BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO ASSIGNMENT 03

MẠNG NƠ-RON VÀ HỌC BIỂU DIỄN TRÊN DỮ LIỆU BẢNG

Giảng viên hướng dẫn: Trần Đình Quế

Sinh viên thực hiện: Đinh Viết Huy

Mã sinh viên: B23DCCN381

Nhóm lớp: 06

Hà Nội – 2026

MỤC LỤC

CHƯƠNG I. BẢN CHẤT LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON 3
1. Bản chất toán học của Học sâu: Phép hợp các hàm tham số hóa 3
1.1. So sánh mô hình truyền thống và mô hình sâu 3
1.2. Cấu trúc toán học bên trong một nơ-ron và một tầng ẩn 3
2. Phân biệt Đặc trưng tự thiết kế (Hand-crafted Features) và Biểu diễn học được (Learned Representation) 3
3. Chu trình huấn luyện 4 bước 3
4. Đối chiếu cài đặt: NumPy From Scratch so với PyTorch Framework 3
CHƯƠNG II. THIẾT KẾ KIẾN TRÚC MẠNG HỌC SÂU 5 TẦNG VÀ PHÂN TÍCH TENSOR SHAPES 9
1. Tổng quan kiến trúc mạng MLP 5 tầng (5-Layer Deep Neural Network Architecture) 9
2. Bảng đặc tả chi tiết kích thước Tensor và Không gian tham số 9
3. Cấu hình chi tiết mạng 5 tầng cho 3 bài toán thực tế 10
3.1. Bài toán 1 - Chẩn đoán bệnh tiểu đường (Diabetes Prediction - Binary Classification) 10
3.2. Bài toán 2 - Định giá bất động sản (House Price Prediction - Regression) 10
3.3. Bài toán 3 - Phân loại mức độ quan tâm khách hàng (Customer Interest - Multiclass Classification) 10
3.4. Giải thích bản chất phân cấp biểu diễn (Hierarchical Representation Learning) 10
CHƯƠNG III. TIẾN HÀNH THỰC NGHIỆM ĐỐI SÁNH 13
A. Bài toán 1 Diabetes Prediction 13
1. Thiết lập thực nghiệm và quy trình triển khai từng bước 13
B. Bài toán 2 House Price Prediction 16
1. Thiết lập thực nghiệm hồi quy từng bước 16

C. Bài toán 3 Customer Interest Prediction 19
1. Thiết lập thực nghiệm đa lớp từng bước 19
CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC 24
A. BÀI TOÁN 1: DIABETES PREDICTION 24
1. Phân tích biểu đồ đường cong huấn luyện 24
2. Phân tích Ma trận nhầm lẫn (Confusion Matrix) trên Tập kiểm thử 24
3. So sánh khoa học giữa ba mô hình trong bài toán Diabetes 24
B. BÀI TOÁN 2: HOUSE PRICE 28
1. Phân tích biểu đồ đường cong huấn luyện hồi quy 28
2. Phân tích đồ thị tương quan giữa Giá thực tế và Giá dự đoán (Scatter Plot) 28
3. Lý giải nguyên nhân sai số thực tế (MAE, RMSE) và giải pháp khắc phục 28
C. BÀI TOÁN 3: CUSTOMER INTEREST PREDICTION 32
1. Phân tích đường cong huấn luyện Cross-Entropy 32
2. Phân tích Ma trận nhầm lẫn đa lớp (Confusion Matrix) 32
3. Thảo luận khoa học về hiệu quả Deep Learning trên dữ liệu khách hàng 32
CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ TOÀN DIỆN 37
1. Tổng kết kết quả thực nghiệm qua 3 bài toán nghiên cứu 37
2. Đúc kết các quy luật khoa học về Mạng nơ-ron sâu (Core Technical Insights) .. 37
3. Hạn chế của hệ thống và Hướng phát triển tiếp theo 37

CHƯƠNG I. BẢN CHẤT LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON

1. Bản chất toán học của Học sâu: Phép hợp các hàm tham số hóa

Deep Learning là phép hợp nhiều hàm tham số hóa. Một neuron tính $z = w^T x + b$ và $a = f(z)$. Với tầng l: z^(l)=W^(l)a^(l-1)+b^(l), a^(l)=ReLU(z^(l)).

1.1. So sánh mô hình truyền thống và mô hình sâu

1.2. Cấu trúc toán học bên trong một nơ-ron và một tầng ẩn

2. Phân biệt Đặc trưng tự thiết kế (Hand-crafted Features) và Biểu diễn học được (Learned Representation)

Đặc trưng tự thiết kế do người xây dựng lựa chọn; biểu diễn học được là vector ẩn h=ReLU(Wx+b) được tối ưu đồng thời với mục tiêu dự đoán.

3. Chu trình huấn luyện 4 bước

Mini-batch → Forward → Loss → Backward theo chain rule → Adam step. BCE dùng cho nhị phân, CrossEntropyLoss cho đa lớp và MSE cho hồi quy.

4. Đối chiếu cài đặt: NumPy From Scratch so với PyTorch Framework

NumPy làm rõ phép toán ma trận và gradient; PyTorch tự động hóa autograd, DataLoader và optimizer. Notebook cell [3] hiển thị trực tiếp models.py và training.py.

[3]:
```
# Đinh Viết Huy - B23DCCN381
print('Cell [3] model-and-training-source: mã thật dùng trong thí nghiệ
print((ROOT / 'src' / 'assignment03' / 'models.py').read_text(encoding=
print((ROOT / 'src' / 'assignment03' / 'training.py').read_text(encodin
```

```
Cell [3] model-and-training-source: mã thật dùng trong thí nghiệm
"""PyTorch model definitions used by Assignment 03."""

from collections.abc import Sequence

from torch import Tensor, nn


class MLP(nn.Module):
    """A feed-forward network with ReLU hidden transformations.

    ``hidden_dims=(h1, h2, h3, h4)`` creates the required five trainable
    Linear layers when combined with the output layer.
    """

    def __init__(
        self,
        input_dim: int,
        hidden_dims: Sequence[int],
        output_dim: int,
    ) -> None:
        super().__init__()
        if input_dim <= 0 or output_dim <= 0:
            raise ValueError("input_dim and output_dim must be positive")
        if any(hidden_dim <= 0 for hidden_dim in hidden_dims):
            raise ValueError("hidden dimensions must be positive")

        dimensions = [input_dim, *hidden_dims, output_dim]
        layers: list[nn.Module] = []
        for index, (in_features, out_features) in enumerate(zip(dimensions, di
mensions[1:])):
            layers.append(nn.Linear(in_features, out_features))
            if index < len(dimensions) - 2:
                layers.append(nn.ReLU())
        self.network = nn.Sequential(*layers)

    def forward(self, inputs: Tensor) -> Tensor:
        return self.network(inputs)


def count_trainable_parameters(model: nn.Module) -> int:
    """Return the number of parameters updated during training."""

    return sum(parameter.numel() for parameter in model.parameters() if parame
ter.requires_grad)

"""Small, reusable PyTorch training loop."""
```

Hình I.1. Executed notebook: 01_diabetes_deep_learning-cell-03-part-01

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [3] trong JupyterLab đã chạy; cell này hiển thị trực tiếp mã nguồn model và vòng lặp huấn luyện được sử dụng.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

```
   small, reusable PyTorch training loop.

import torch
from torch import Tensor, nn
from torch.utils.data import DataLoader, TensorDataset

from .config import TrainingConfig, set_seed


def _make_loader(features: Tensor, targets: Tensor, config: TrainingConfig) ->
DataLoader:
    return DataLoader(
        TensorDataset(features, targets),
        batch_size=config.batch_size,
        shuffle=True,
        generator=torch.Generator().manual_seed(config.seed),
    )


def _mean_loss(model: nn.Module, loader: DataLoader, criterion: nn.Module, dev
ice: torch.device) -> float:
    model.eval()
    total_loss = 0.0
    total_items = 0
    with torch.no_grad():
        for features, targets in loader:
            predictions = model(features.to(device))
            loss = criterion(predictions, targets.to(device))
            item_count = features.shape[0]
            total_loss += loss.item() * item_count
            total_items += item_count
    return total_loss / total_items if total_items else 0.0


def train_model(
    model: nn.Module,
    train_features: Tensor,
    train_targets: Tensor,
    *,
    validation_features: Tensor | None = None,
    validation_targets: Tensor | None = None,
    criterion: nn.Module | None = None,
    config: TrainingConfig | None = None,
) -> dict[str, list[float]]:
    """Train a model and return epoch-level training/validation loss histor
y."""

    config = config or TrainingConfig()
    set_seed(config.seed)
    device = torch.device(config.device)
    model.to(device)
    criterion = criterion or nn.MSELoss()
    optimizer_name = config.optimizer.lower()
    if optimizer_name == "adam":
```

Hình I.2. Executed notebook: 01_diabetes_deep_learning-cell-03-part-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [3] trong JupyterLab đã chạy; cell này hiển thị trực tiếp mã nguồn model và vòng lặp huấn luyện được sử dụng.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

```
            optimizer = torch.optim.Adam(model.parameters(), lr=config.learning_ra
te)
        elif optimizer_name == "sgd":
            optimizer = torch.optim.SGD(model.parameters(), lr=config.learning_rat
e)
        else:
            raise ValueError(f"unsupported optimizer: {config.optimizer}")

        train_loader = _make_loader(train_features, train_targets, config)
        validation_loader = None
        if validation_features is not None or validation_targets is not None:
            if validation_features is None or validation_targets is None:
                raise ValueError("validation features and targets must be provided
together")
            validation_loader = _make_loader(validation_features, validation_targe
ts, config)

        history: dict[str, list[float]] = {"train_loss": [], "validation_loss":
[]}
        for _ in range(config.epochs):
            model.train()
            for features, targets in train_loader:
                optimizer.zero_grad(set_to_none=True)
                predictions = model(features.to(device))
                loss = criterion(predictions, targets.to(device))
                loss.backward()
                optimizer.step()
            history["train_loss"].append(_mean_loss(model, train_loader, criterio
n, device))
            if validation_loader is not None:
                history["validation_loss"].append(_mean_loss(model, validation_loa
der, criterion, device))
        return history
```

Hình I.3. Executed notebook: 01_diabetes_deep_learning-cell-03-part-03

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [3] trong JupyterLab đã chạy; cell này hiển thị trực tiếp mã nguồn model và vòng lặp huấn luyện được sử dụng.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

CHƯƠNG II. THIẾT KẾ KIẾN TRÚC MẠNG HỌC SÂU 5 TẦNG VÀ PHÂN TÍCH TENSOR SHAPES

1. Tổng quan kiến trúc mạng MLP 5 tầng (5-Layer Deep Neural Network Architecture)

Kiến trúc dùng năm Linear: input → 128 → 64 → 32 → 16 → output. Bốn tầng ẩn dùng ReLU; tầng cuối sinh logit hoặc giá trị hồi quy.

2. Bảng đặc tả chi tiết kích thước Tensor và Không gian tham số

Bài toán	Input	Hidden dims	Output	Tham số
Dự đoán bệnh tiểu đường	76	128→64→32→16	1	20,737
Dự đoán giá nhà	293	128→64→32→16	1	48,513
Phân loại mức độ quan tâm khách hàng	3003	128→64→32→16	5	395,461

Số tham số Linear bằng in_features×out_features+out_features. Customer interest có 3003 chiều nên lớp đầu tiên làm số tham số tăng mạnh.

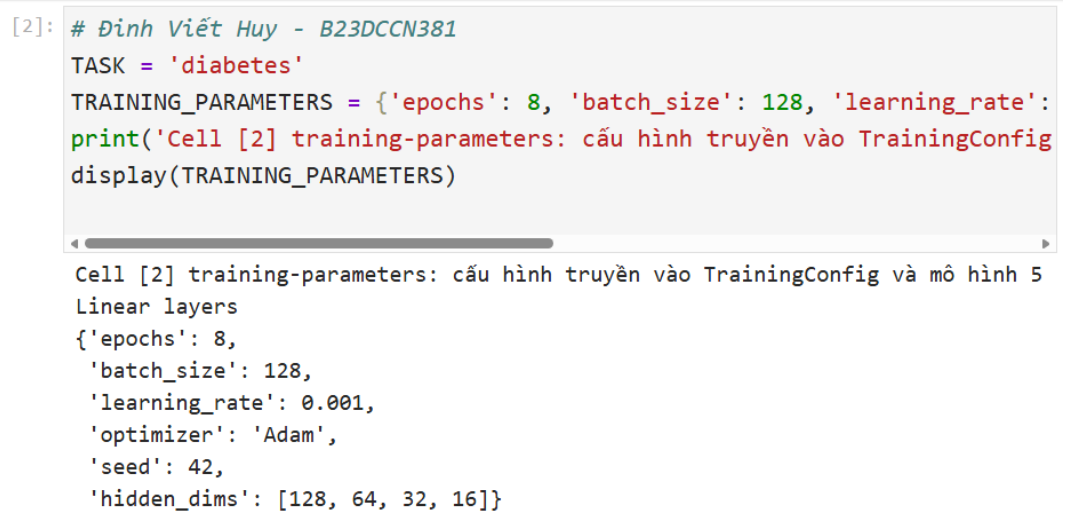
3. Cấu hình chi tiết mạng 5 tầng cho 3 bài toán thực tế

3.1. Bài toán 1 - Chẩn đoán bệnh tiểu đường (Diabetes Prediction - Binary Classification)

3.2. Bài toán 2 - Định giá bất động sản (House Price Prediction - Regression)

3.3. Bài toán 3 - Phân loại mức độ quan tâm khách hàng (Customer Interest - Multiclass Classification)

3.4. Giải thích bản chất phân cấp biểu diễn (Hierarchical Representation Learning)



```
[2]: # Đinh Viết Huy - B23DCCN381
TASK = 'diabetes'
TRAINING_PARAMETERS = {'epochs': 8, 'batch_size': 128, 'learning_rate':
print('Cell [2] training-parameters: cấu hình truyền vào TrainingConfig
display(TRAINING_PARAMETERS)

Cell [2] training-parameters: cấu hình truyền vào TrainingConfig và mô hình 5
Linear layers
{'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'optimizer': 'Adam',
 'seed': 42,
 'hidden_dims': [128, 64, 32, 16]}
```

Hình II.1. Executed notebook: 01_diabetes_deep_learning-cell-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [2] trong JupyterLab đã chạy; cell này khai báo epochs, batch size, learning rate, optimizer, seed và hidden dimensions.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

[2]:
```
# Đinh Viết Huy - B23DCCN381
TASK = 'house_price'
TRAINING_PARAMETERS = {'epochs': 8, 'batch_size': 128, 'learning_rate':
print('Cell [2] training-parameters: cấu hình truyền vào TrainingConfig
display(TRAINING_PARAMETERS)
```

```
Cell [2] training-parameters: cấu hình truyền vào TrainingConfig và mô hình 5
Linear layers
{'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'optimizer': 'Adam',
 'seed': 42,
 'hidden_dims': [128, 64, 32, 16]}
```

Hình II.2. Executed notebook: 02_house_price_deep_learning-cell-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [2] trong JupyterLab đã chạy; cell này khai báo epochs, batch size, learning rate, optimizer, seed và hidden dimensions.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

[2]:
```
# Đinh Viết Huy - B23DCCN381
TASK = 'customer_interest'
TRAINING_PARAMETERS = {'epochs': 8, 'batch_size': 128, 'learning_rate':
print('Cell [2] training-parameters: cấu hình truyền vào TrainingConfig
display(TRAINING_PARAMETERS)
```

```
Cell [2] training-parameters: cấu hình truyền vào TrainingConfig và mô hình 5
Linear layers
{'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'optimizer': 'Adam',
 'seed': 42,
 'hidden_dims': [128, 64, 32, 16]}
```

Hình II.3. Executed notebook: 03_customer_interest_deep_learning-cell-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [2] trong JupyterLab đã chạy; cell này khai báo epochs, batch size, learning rate, optimizer, seed và hidden dimensions.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

CHƯƠNG III. TIẾN HÀNH THỰC NGHIỆM ĐỐI SÁNH

Trong cả ba notebook, cell [2] khai báo epochs=8, batch_size=128, learning_rate=0,001, optimizer=Adam, seed=42 và hidden_dims=[128,64,32,16]. Cell [3] hiển thị mã model/training; cell [4] đọc metadata/results; cell [5] hiển thị biểu đồ; cell [6] hiển thị kết quả đã lưu.

A. Bài toán 1 Diabetes Prediction

1. Thiết lập thực nghiệm và quy trình triển khai từng bước

Mô hình	Accuracy	Precision	Recall	F1	MAE	RMSE	R^2	Tham số	Giây
MLP cơ sở	0.9573 (95.73%)	0.8478 (84.78%)	0.6094 (60.94%)	0.7091 (70.91%)	-	-	-	4,993	0.505
MLP sâu hơn	0.9560 (95.60%)	0.7719 (77.19%)	0.6875 (68.75%)	0.7273 (72.73%)	-	-	-	7,041	0.574
MLP 5 lớp	0.9507 (95.07%)	0.7455 (74.55%)	0.6406 (64.06%)	0.6891 (68.91%)	-	-	-	20,737	0.673

Theo F1, MLP sâu hơn tốt nhất với accuracy=0.9560, precision=0.7719, recall=0.6875, F1=0.7273.

[4]:
```
# Đinh Viết Huy - B23DCCN381
metadata_path = ROOT / 'artifacts' / TASK / 'metadata.json'
results_path = ROOT / 'artifacts' / TASK / 'results.json'
bundle = SimpleNamespace(metadata=json.loads(metadata_path.read_text(en
experiment_results = json.loads(results_path.read_text(encoding='utf-8'
print('Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau
display(bundle.metadata)
```

```
Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau training
{'task': 'diabetes',
 'task_type': 'binary',
 'input_dim': 76,
 'output_dim': 1,
 'hidden_dims': [128, 64, 32, 16],
 'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'parameter_count': 20737,
 'metrics': {'accuracy': 0.9506666666666667,
  'precision': 0.7454545454545455,
  'recall': 0.640625,
  'f1': 0.6890756302521008,
  'confusion_matrix': [[672, 14], [23, 41]]},
 'history': {'train_loss': [0.6516214665004185,
   0.22391879095349992,
   0.1657770406263215,
   0.14301338020392826,
   0.13040573222722326,
   0.12284162046228136,
   0.11454919686061996,
   0.1010205169575555],
  'validation_loss': [0.6519008933703104,
   0.22762743854522705,
   0.17157967058817544,
   0.14987069044510523,
   0.13889234828948974,
   0.13436228581269583,
   0.13109238219261168,
   0.12424243384599686]},
 'training_seconds': 0.6733527999999751,
 'data_metadata': {'target': 'diabetes'}}
```

Hình III-A.1. Executed notebook: 01_diabetes_deep_learning-cell-04

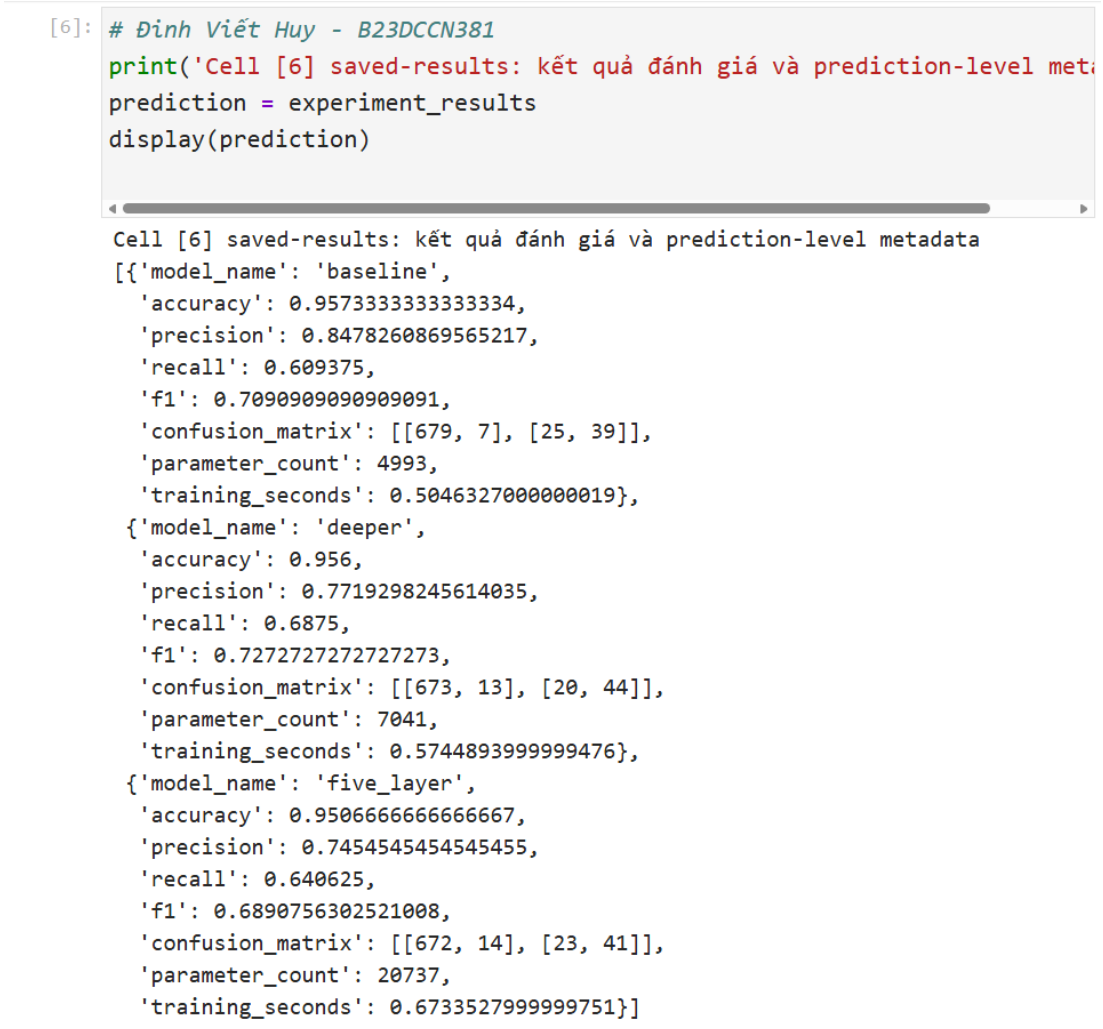
Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [4] trong JupyterLab đã chạy; cell này đọc metadata và metric thật đã lưu sau huấn luyện.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



```
[6]: # Đinh Viết Huy - B23DCCN381
print('Cell [6] saved-results: kết quả đánh giá và prediction-level meta
prediction = experiment_results
display(prediction)
```

```
Cell [6] saved-results: kết quả đánh giá và prediction-level metadata
[{'model_name': 'baseline',
  'accuracy': 0.9573333333333334,
  'precision': 0.8478260869565217,
  'recall': 0.609375,
  'f1': 0.7090909090909091,
  'confusion_matrix': [[679, 7], [25, 39]],
  'parameter_count': 4993,
  'training_seconds': 0.5046327000000019},
 {'model_name': 'deeper',
  'accuracy': 0.956,
  'precision': 0.7719298245614035,
  'recall': 0.6875,
  'f1': 0.7272727272727273,
  'confusion_matrix': [[673, 13], [20, 44]],
  'parameter_count': 7041,
  'training_seconds': 0.5744893999999476},
 {'model_name': 'five_layer',
  'accuracy': 0.9506666666666667,
  'precision': 0.7454545454545455,
  'recall': 0.640625,
  'f1': 0.6890756302521008,
  'confusion_matrix': [[672, 14], [23, 41]],
  'parameter_count': 20737,
  'training_seconds': 0.6733527999999751}]
```

Hình III-A.2. Executed notebook: 01_diabetes_deep_learning-cell-06

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [6] trong JupyterLab đã chạy; cell này hiển thị toàn bộ kết quả đánh giá đã lưu của ba mô hình.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

B. Bài toán 2 House Price Prediction

1. Thiết lập thực nghiệm hồi quy từng bước

Mô hình	Accuracy	Precision	Recall	F1	MAE	RMSE	R^2	Tham số	Giây
MLP cơ sở	-	-	-	-	2.7815	2.7944	-43.6293	18,881	0.116
MLP sâu hơn	-	-	-	-	0.4905	0.5487	-0.7204	20,929	0.122
MLP 5 lớp	-	-	-	-	1.7042	1.7300	-16.1053	48,513	0.146

Theo RMSE, MLP sâu hơn tốt nhất với MAE=0.4905, RMSE=0.5487, R^2=-0.7204. R^2 âm là tín hiệu mô hình hiện chưa vượt baseline trung bình.

[4]:
```
# Đinh Viết Huy - B23DCCN381
metadata_path = ROOT / 'artifacts' / TASK / 'metadata.json'
results_path = ROOT / 'artifacts' / TASK / 'results.json'
bundle = SimpleNamespace(metadata=json.loads(metadata_path.read_text(en
experiment_results = json.loads(results_path.read_text(encoding='utf-8'
print('Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau
display(bundle.metadata)
```

```
Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau training
{'task': 'house_price',
 'task_type': 'regression',
 'input_dim': 293,
 'output_dim': 1,
 'hidden_dims': [128, 64, 32, 16],
 'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'parameter_count': 48513,
 'metrics': {'mae': 1.7041923099093967,
  'rmse': 1.7299891227930813,
  'r2': -16.1052763207004},
 'history': {'train_loss': [140.39231247674851,
   133.30755687895277,
   113.58654000418527,
   69.36248459588914,
   8.248717353457497,
   14.024039540972028,
   0.16958214810916356,
   2.9954516229175385],
  'validation_loss': [140.40258958604602,
   133.27002699110244,
   113.48428327772352,
   69.23941023084852,
   8.21463885837131,
   14.280777719285753,
   0.21199606590800815,
   2.9483578734927707]},
 'training_seconds': 0.14607759999989867,
 'data_metadata': {'target': 'SalePrice', 'target_transform': 'log1p'}}
```

Hình III-B.1. Executed notebook: 02_house_price_deep_learning-cell-04

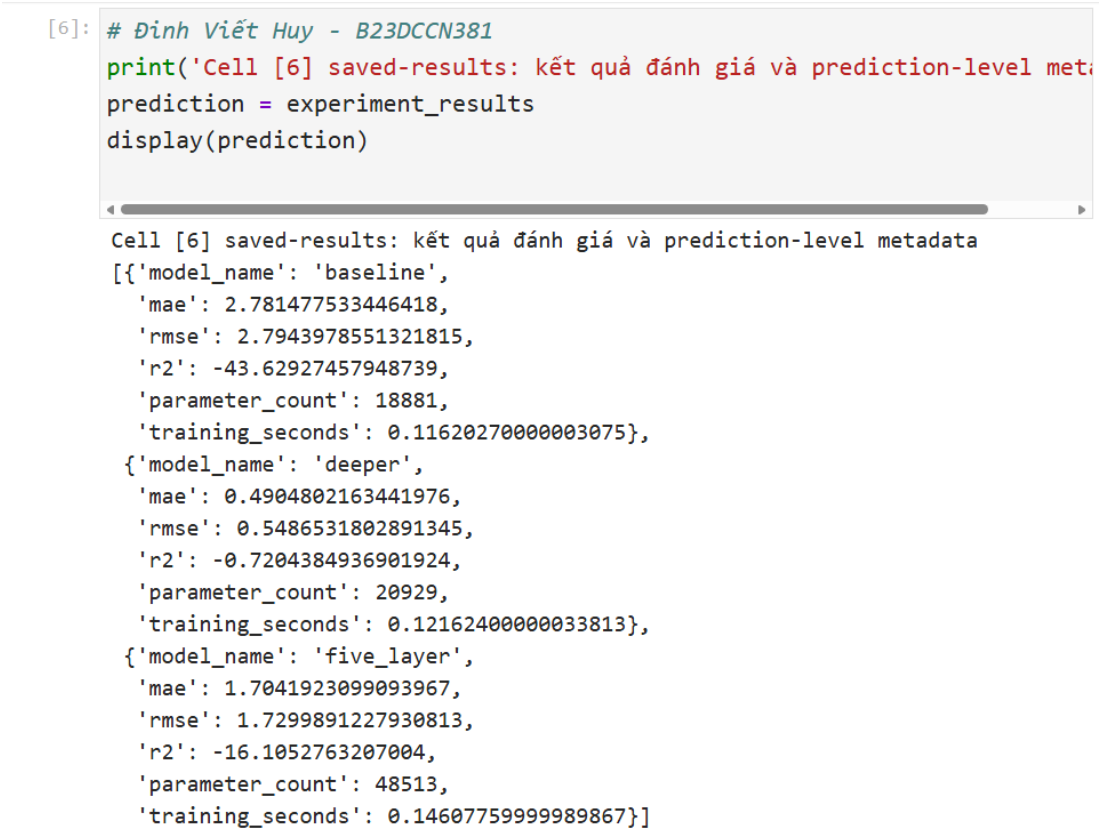
Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [4] trong JupyterLab đã chạy; cell này đọc metadata và metric thật đã lưu sau huấn luyện.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



```
[6]: # Đinh Viết Huy - B23DCCN381
print('Cell [6] saved-results: kết quả đánh giá và prediction-level meta
prediction = experiment_results
display(prediction)
```

```
Cell [6] saved-results: kết quả đánh giá và prediction-level metadata
[{'model_name': 'baseline',
  'mae': 2.781477533446418,
  'rmse': 2.7943978551321815,
  'r2': -43.62927457948739,
  'parameter_count': 18881,
  'training_seconds': 0.11620270000003075},
 {'model_name': 'deeper',
  'mae': 0.4904802163441976,
  'rmse': 0.5486531802891345,
  'r2': -0.7204384936901924,
  'parameter_count': 20929,
  'training_seconds': 0.12162400000033813},
 {'model_name': 'five_layer',
  'mae': 1.7041923099093967,
  'rmse': 1.7299891227930813,
  'r2': -16.1052763207004,
  'parameter_count': 48513,
  'training_seconds': 0.14607759999989867}]
```

Hình III-B.2. Executed notebook: 02_house_price_deep_learning-cell-06

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [6] trong JupyterLab đã chạy; cell này hiển thị toàn bộ kết quả đánh giá đã lưu của ba mô hình.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

C. Bài toán 3 Customer Interest Prediction

1. Thiết lập thực nghiệm đa lớp từng bước

Mô hình	Accuracy	Precision	Recall	F1	MAE	RMSE	R^2	Tham số	Giây
MLP cơ sở	0.7467 (74.67%)	0.8512 (85.12%)	0.5091 (50.91%)	0.4978 (49.78%)	-	-	-	192,581	0.821
MLP sâu hơn	0.7489 (74.89%)	0.5615 (56.15%)	0.5256 (52.56%)	0.5110 (51.10%)	-	-	-	194,501	0.923
MLP 5 lớp	0.7111 (71.11%)	0.4104 (41.04%)	0.4822 (48.22%)	0.4412 (44.12%)	-	-	-	395,461	1.153

Theo F1, MLP sâu hơn tốt nhất với accuracy=0.7489, precision=0.5615, recall=0.5256, F1=0.5110.

[4]:
```
# Đinh Viết Huy - B23DCCN381
metadata_path = ROOT / 'artifacts' / TASK / 'metadata.json'
results_path = ROOT / 'artifacts' / TASK / 'results.json'
bundle = SimpleNamespace(metadata=json.loads(metadata_path.read_text(en
experiment_results = json.loads(results_path.read_text(encoding='utf-8'
print('Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau
display(bundle.metadata)
```

```
Cell [4] artifact-metadata: metadata và metrics thật đã lưu sau training
{'task': 'customer_interest',
 'task_type': 'multiclass',
 'input_dim': 3003,
 'output_dim': 5,
 'hidden_dims': [128, 64, 32, 16],
 'epochs': 8,
 'batch_size': 128,
 'learning_rate': 0.001,
 'parameter_count': 395461,
 'metrics': {'accuracy': 0.7111111111111111,
  'precision': 0.4104380293360741,
  'recall': 0.4822269271777645,
  'f1': 0.44115879555453824,
  'confusion_matrix': [[112, 15, 0, 0, 19],
   [8, 201, 0, 0, 33],
   [39, 11, 0, 0, 21],
   [20, 5, 0, 0, 14],
   [35, 40, 0, 0, 327]]},
 'history': {'train_loss': [1.4375672535669235,
   1.3126157222475325,
   1.242342246827625,
   1.041288436480931,
   0.7555046224594116,
   0.6181172291437785,
   0.5187707503636678,
   0.37381609042485553],
  'validation_loss': [1.4412355136871338,
   1.3315679767396715,
   1.2861590925852457,
   1.1448967785305446,
   0.9631740562121074,
   0.924545500808292,
   0.9323519840505388,
   1.0146001575390498]},
 'training_seconds': 1.1531119999999646,
 'data_metadata': {'target': 'Category',
  'classes': ['Bottoms', 'Dresses', 'Intimate', 'Jackets', 'Tops']}}
```

Hình III-C.1. Executed notebook: 03_customer_interest_deep_learning-cell-04

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [4] trong JupyterLab đã chạy; cell này đọc metadata và metric thật đã lưu sau huấn luyện.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

[6]:
```
# Đinh Viết Huy - B23DCCN381
print('Cell [6] saved-results: kết quả đánh giá và prediction-level meta
prediction = experiment_results
display(prediction)
```

```
Cell [6] saved-results: kết quả đánh giá và prediction-level metadata
[{'model_name': 'baseline',
  'accuracy': 0.7466666666666667,
  'precision': 0.8511774593986429,
  'recall': 0.5090638980858834,
  'f1': 0.49775276546226266,
  'confusion_matrix': [[115, 15, 0, 0, 16],
   [1, 191, 0, 0, 50],
   [22, 8, 3, 0, 38],
   [4, 7, 0, 1, 27],
   [8, 32, 0, 0, 362]],
  'parameter_count': 192581,
  'training_seconds': 0.8211228999998639},
 {'model_name': 'deeper',
  'accuracy': 0.7488888888888889,
  'precision': 0.5614733533726904,
  'recall': 0.5256486489473153,
  'f1': 0.5110483496210109,
  'confusion_matrix': [[123, 13, 0, 0, 10],
   [6, 196, 0, 0, 40],
   [23, 9, 8, 0, 31],
   [4, 4, 5, 0, 26],
   [14, 40, 1, 0, 347]],
  'parameter_count': 194501,
  'training_seconds': 0.922945300000265},
 {'model_name': 'five_layer',
  'accuracy': 0.7111111111111111,
  'precision': 0.4104380293360741,
  'recall': 0.4822269271777645,
  'f1': 0.44115879555453824,
  'confusion_matrix': [[112, 15, 0, 0, 19],
   [8, 201, 0, 0, 33],
   [39, 11, 0, 0, 21],
   [20, 5, 0, 0, 14],
   [35, 40, 0, 0, 327]],
  'parameter_count': 395461,
  'training_seconds': 1.1531119999999646}]
```

Hình III-C.2. Executed notebook: 03_customer_interest_deep_learning-cell-06

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [6] trong JupyterLab đã chạy; cell này hiển thị toàn bộ kết quả đánh giá đã lưu của ba mô hình.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC

A. BÀI TOÁN 1: DIABETES PREDICTION

1. Phân tích biểu đồ đường cong huấn luyện

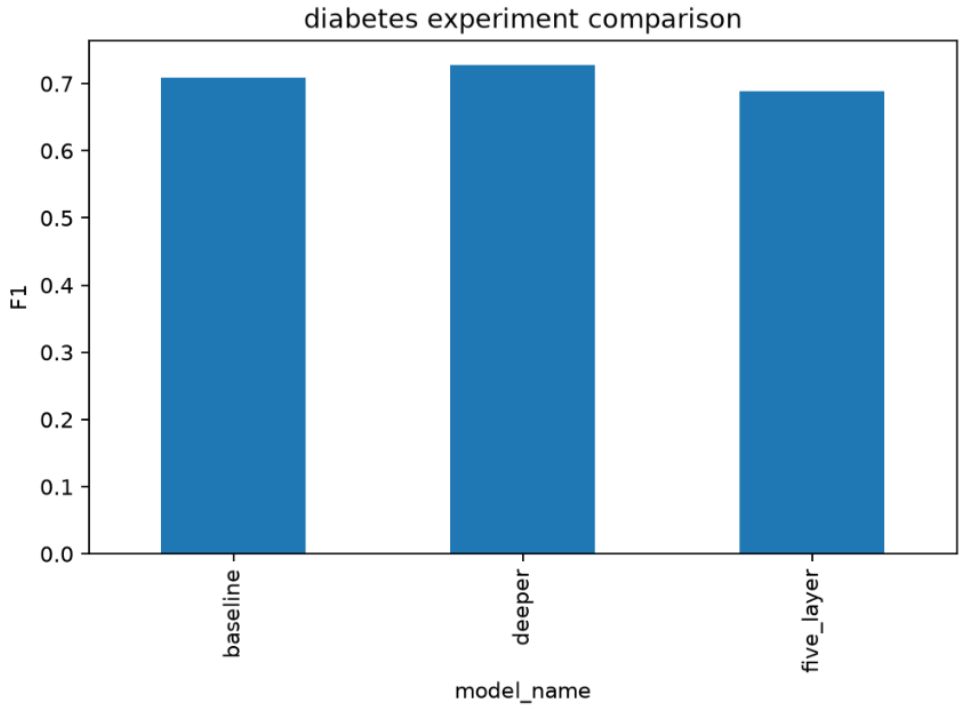
2. Phân tích Ma trận nhầm lẫn (Confusion Matrix) trên Tập kiểm thử

3. So sánh khoa học giữa ba mô hình trong bài toán Diabetes

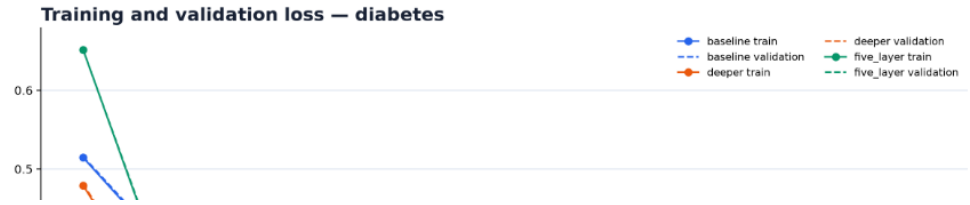
Đường loss phản ánh tối ưu theo epoch; confusion matrix đọc theo nhãn thật ở hàng và nhãn dự đoán ở cột; scatter hồi quy đọc khoảng cách điểm tới đường y=ŷ. Cần đối chiếu trực tiếp cell [5] và metric cell [4].

[5]:
```
# Đinh Viết Huy - B23DCCN381
chart_paths = [
    ROOT / 'artifacts' / TASK / 'model_comparison.png',
    ROOT / 'report' / 'evidence' / 'loss_curves' / f'loss_{TASK}.png',
    ROOT / 'report' / 'evidence' / 'diagrams' / 'architecture_tensor_sh
    ROOT / 'report' / 'evidence' / 'diagrams' / 'training_data_flow.png
]
if TASK == 'house_price':
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'regression' /
else:
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'confusion_mat
print('Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của
for chart_path in chart_paths:
    if chart_path.is_file():
        print(chart_path.relative_to(ROOT))
        display(Image(filename=str(chart_path), width=900))
```

```
Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của task diabetes
artifacts\diabetes\model_comparison.png
```




```
report\evidence\loss_curves\loss_diabetes.png
```



Hình IV-A.1. Executed notebook: 01_diabetes_deep_learning-cell-05-part-01

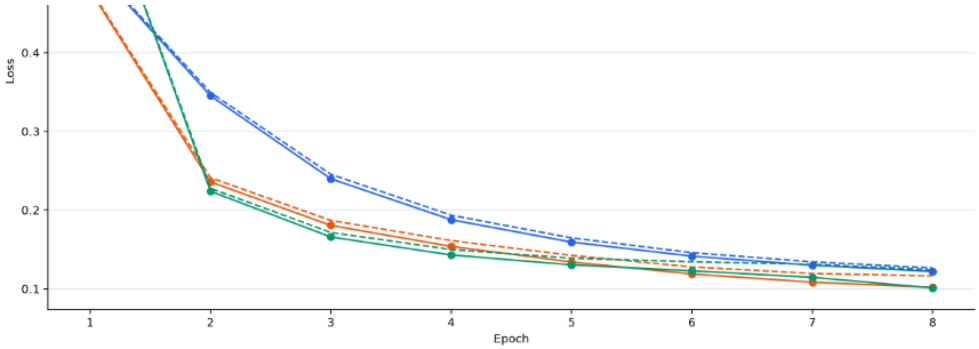
Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

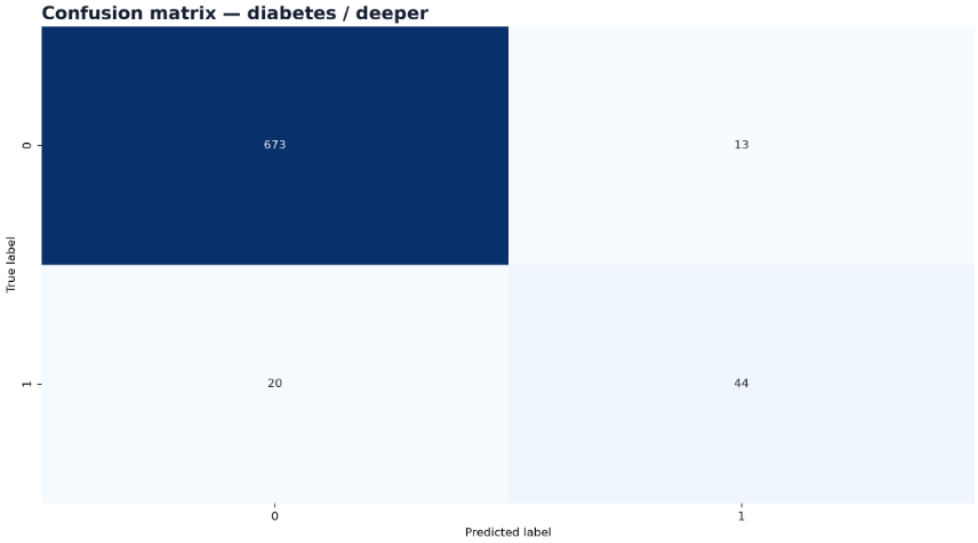
Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

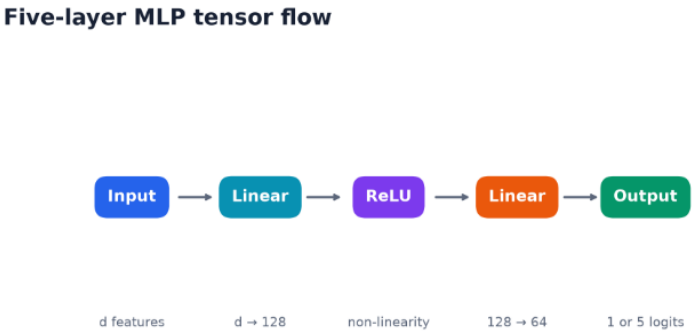
Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



report\evidence\confusion_matrices\confusion_diabetes.png



report\evidence\diagrams\architecture_tensor_shapes.png



report\evidence\diagrams\training_data_flow.png



Hình IV-A.2. Executed notebook: 01_diabetes_deep_learning-cell-05-part-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

B. BÀI TOÁN 2: HOUSE PRICE

1. Phân tích biểu đồ đường cong huấn luyện hồi quy

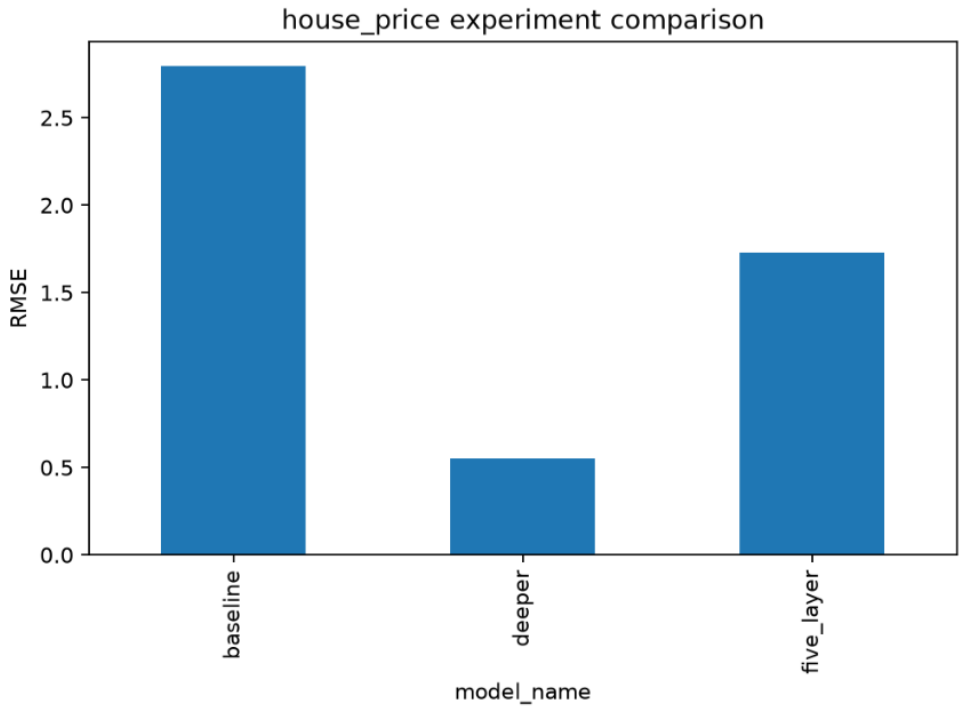
2. Phân tích đồ thị tương quan giữa Giá thực tế và Giá dự đoán (Scatter Plot)

3. Lý giải nguyên nhân sai số thực tế (MAE, RMSE) và giải pháp khắc phục

Đường loss phản ánh tối ưu theo epoch; confusion matrix đọc theo nhãn thật ở hàng và nhãn dự đoán ở cột; scatter hồi quy đọc khoảng cách điểm tới đường y=ŷ. Cần đối chiếu trực tiếp cell [5] và metric cell [4].

[5]:
```
# Đinh Viết Huy - B23DCCN381
chart_paths = [
    ROOT / 'artifacts' / TASK / 'model_comparison.png',
    ROOT / 'report' / 'evidence' / 'loss_curves' / f'loss_{TASK}.png',
    ROOT / 'report' / 'evidence' / 'diagrams' / 'architecture_tensor_sh
    ROOT / 'report' / 'evidence' / 'diagrams' / 'training_data_flow.png
]
if TASK == 'house_price':
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'regression' /
else:
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'confusion_matr
print('Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của t
for chart_path in chart_paths:
    if chart_path.is_file():
        print(chart_path.relative_to(ROOT))
        display(Image(filename=str(chart_path), width=900))
```

```
Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của task house_pri
ce
artifacts\house_price\model_comparison.png
```




```
report\evidence\loss_curves\loss_house_price.png
```



Hình IV-B.1. Executed notebook: 02_house_price_deep_learning-cell-05-part-01

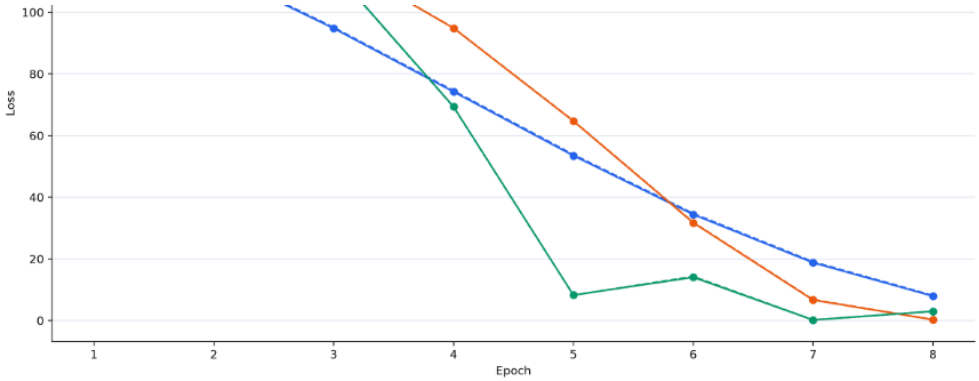
Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

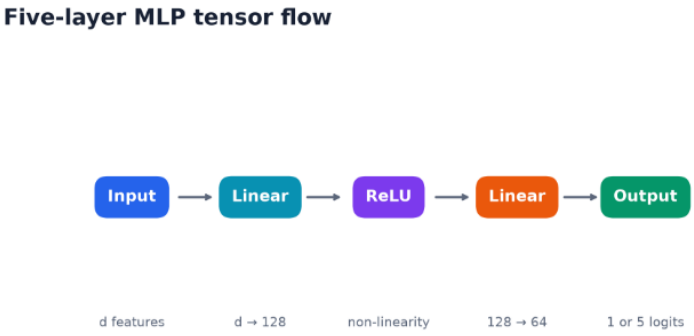
Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



report\evidence\regression\regression_actual_vs_predicted.png



report\evidence\diagrams\architecture_tensor_shapes.png



report\evidence\diagrams\training_data_flow.png

Hình IV-B.2. Executed notebook: 02_house_price_deep_learning-cell-05-part-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

C. BÀI TOÁN 3: CUSTOMER INTEREST PREDICTION

1. Phân tích đường cong huấn luyện Cross-Entropy

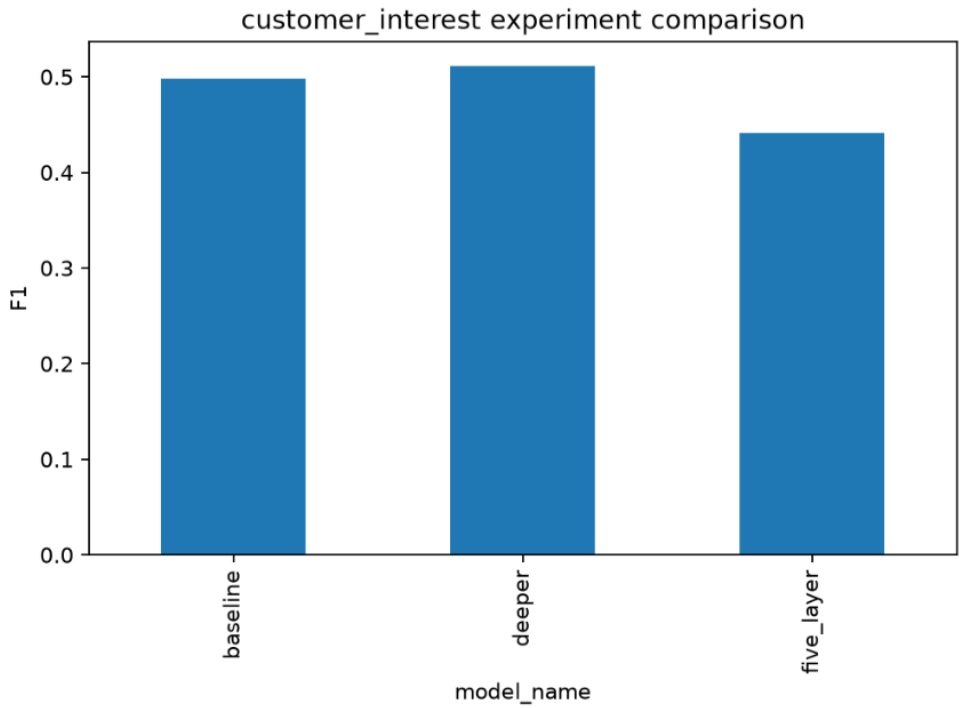
2. Phân tích Ma trận nhầm lẫn đa lớp (Confusion Matrix)

3. Thảo luận khoa học về hiệu quả Deep Learning trên dữ liệu khách hàng

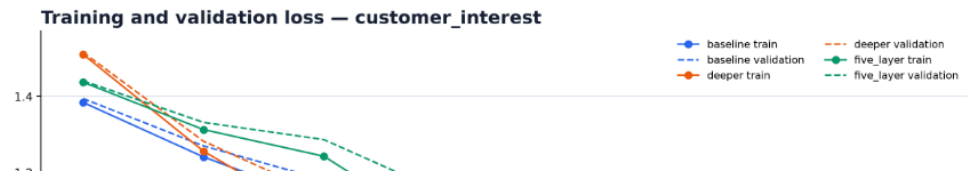
Đường loss phản ánh tối ưu theo epoch; confusion matrix đọc theo nhãn thật ở hàng và nhãn dự đoán ở cột; scatter hồi quy đọc khoảng cách điểm tới đường y=ŷ. Cần đối chiếu trực tiếp cell [5] và metric cell [4].

[5]:
```
# Đinh Viết Huy - B23DCCN381
chart_paths = [
    ROOT / 'artifacts' / TASK / 'model_comparison.png',
    ROOT / 'report' / 'evidence' / 'loss_curves' / f'loss_{TASK}.png',
    ROOT / 'report' / 'evidence' / 'diagrams' / 'architecture_tensor_sh
    ROOT / 'report' / 'evidence' / 'diagrams' / 'training_data_flow.png
]
if TASK == 'house_price':
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'regression' /
else:
    chart_paths.insert(2, ROOT / 'report' / 'evidence' / 'confusion_matr
print('Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của t
for chart_path in chart_paths:
    if chart_path.is_file():
        print(chart_path.relative_to(ROOT))
        display(Image(filename=str(chart_path), width=900))
```

```
Cell [5] charts: biểu đồ và sơ đồ được tạo từ artifact thật của task customer_
interest
artifacts\customer_interest\model_comparison.png
```




```
report\evidence\loss_curves\loss_customer_interest.png
```



Hình IV-C.1. Executed notebook: 03_customer_interest_deep_learning-cell-05-part-01

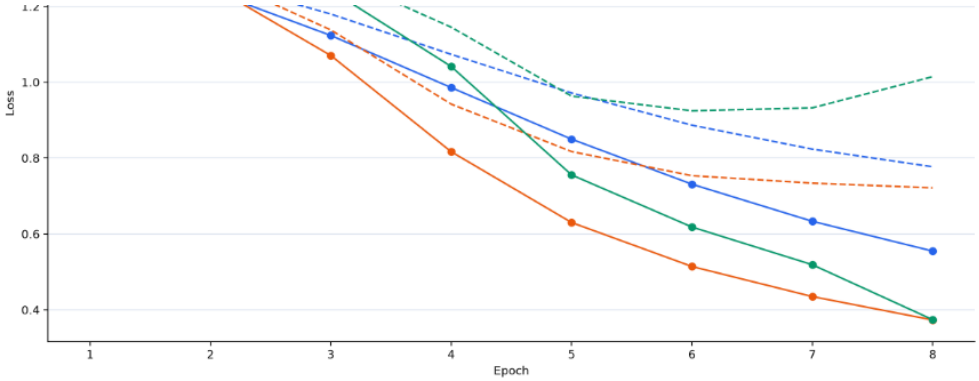
Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

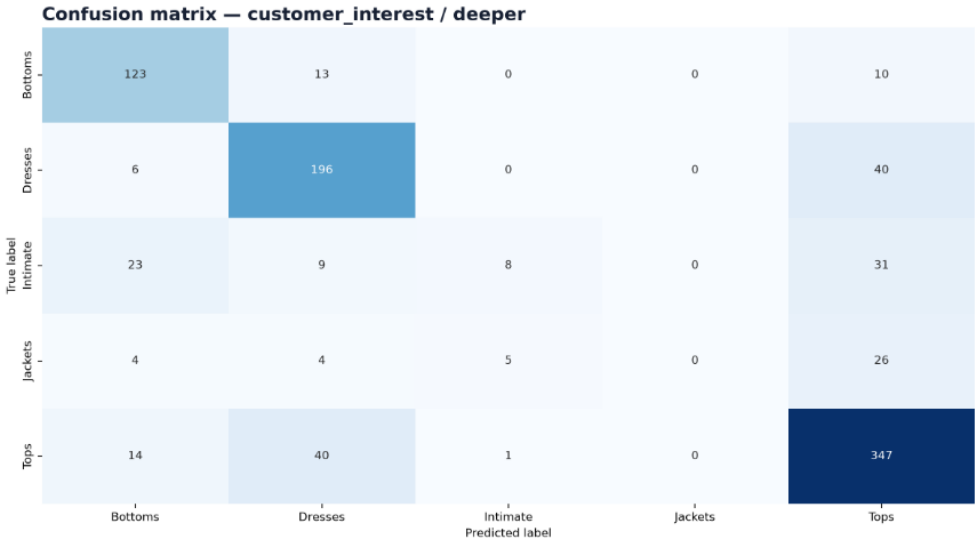
Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

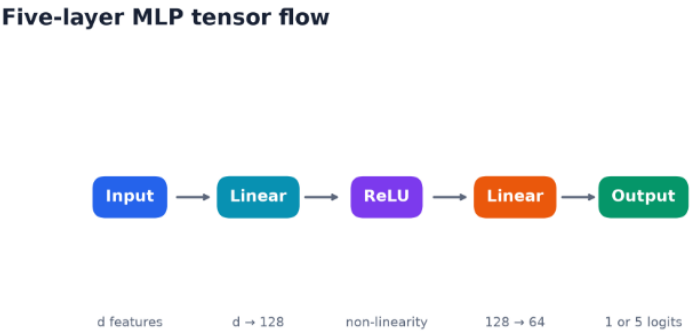
Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



report\evidence\confusion_matrices\confusion_customer_interest.png



report\evidence\diagrams\architecture_tensor_shapes.png



report\evidence\diagrams\training_data_flow.png

Hình IV-C.2. Executed notebook: 03_customer_interest_deep_learning-cell-05-part-02

Nguồn và cell liên quan: ảnh được chụp trực tiếp riêng Cell [5] trong JupyterLab đã chạy; cell này hiển thị biểu đồ so sánh mô hình, loss, ma trận nhầm lẫn hoặc scatter và sơ đồ kiến trúc.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ TOÀN DIỆN

1. Tổng kết kết quả thực nghiệm qua 3 bài toán nghiên cứu

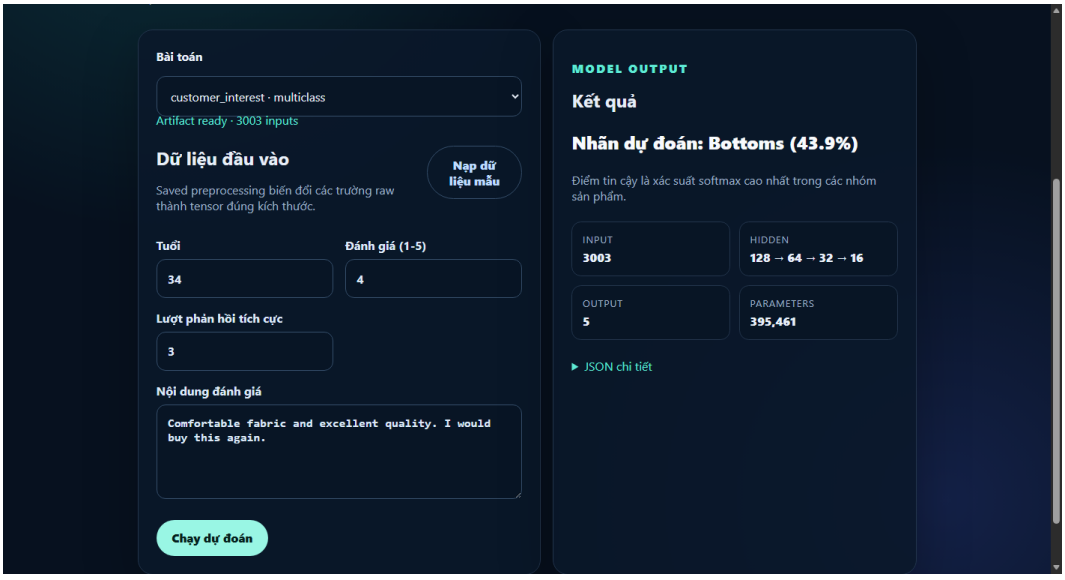
Bài toán	Loại	Input	Tham số	Thời gian train
Dự đoán bệnh tiểu đường	binary	76	20,737	0.673s
Dự đoán giá nhà	regression	293	48,513	0.146s
Phân loại mức độ quan tâm khách hàng	multiclass	3003	395,461	1.153s

2. Đúc kết các quy luật khoa học về Mạng nơ-ron sâu (Core Technical Insights)

Độ sâu chỉ hữu ích khi representation, loss và dữ liệu phù hợp. Accuracy phải đọc cùng F1/recall; hồi quy phải đọc MAE/RMSE/R^2; số tham số phải đọc cùng thời gian train và validation loss.

3. Hạn chế của hệ thống và Hướng phát triển tiếp theo

Protocol hiện dùng một seed và tám epoch. Có thể mở rộng bằng early stopping, class weighting, scheduler, regularization và nhiều lần chia dữ liệu.



Hình V.1. Responsive demo: desktop-customer_interest

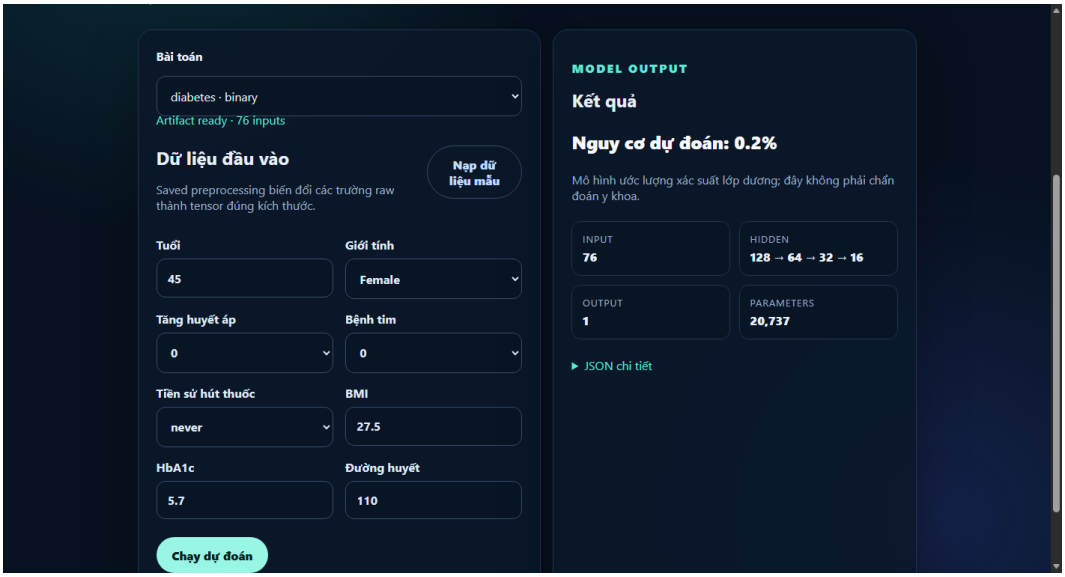
Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



Hình V.2. Responsive demo: desktop-diabetes

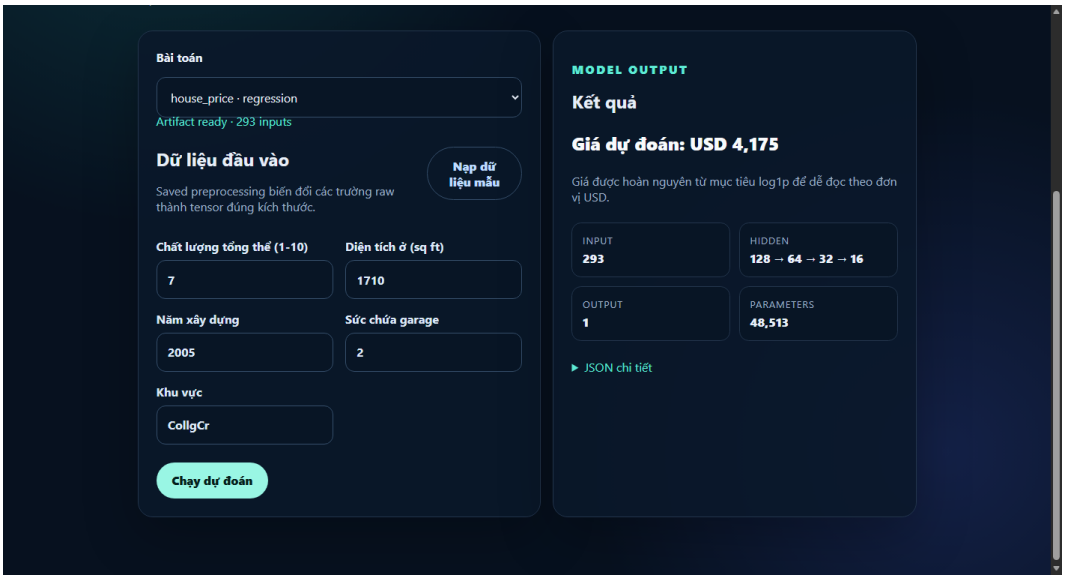
Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.



Hình V.3. Responsive demo: desktop-house_price

Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

PTKHTTM · ASSIGNMENT 03

Raw-input neural network demo

Nhập dữ liệu nghiệp vụ; backend áp dụng saved preprocessing và checkpoint PyTorch trước khi suy luận.

Bài toán

customer_interest · multiclass

Artifact ready · 3003 inputs

Dữ liệu đầu vào

Saved preprocessing biến đổi các trường raw thành tensor đúng kích thước.

Nạp dữ liệu mẫu

Tuổi

34

Đánh giá (1-5)

4

Lượt phản hồi tích cực

3

Hình V.4. Responsive demo: mobile-customer_interest

Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

network demo

Nhập dữ liệu nghiệp vụ; backend áp dụng saved preprocessing và checkpoint PyTorch trước khi suy luận.

Bài toán

diabetes · binary

Artifact ready · 76 inputs

Dữ liệu đầu vào

Saved preprocessing biến đổi các trường raw thành tensor đúng kích thước.

Nạp dữ liệu mẫu

Tuổi

45

Giới tính

Female

Tăng huyết áp

0

Bệnh tim

0

Tiền sử hút thuốc

Hình V.5. Responsive demo: mobile-diabetes

Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.

PTKHTTM · ASSIGNMENT 03

Raw-input neural network demo

Nhập dữ liệu nghiệp vụ; backend áp dụng saved preprocessing và checkpoint PyTorch trước khi suy luận.

Bài toán

house_price · regression

Artifact ready · 293 inputs

Dữ liệu đầu vào

Saved preprocessing biến đổi các trường raw thành tensor đúng kích thước.

Nạp dữ liệu mẫu

Chất lượng tổng thể (1-10)

7

Diện tích ở (sq ft)

1710

Năm xây dựng

2005

Hình V.6. Responsive demo: mobile-house_price

Nguồn: ảnh chụp trực tiếp giao diện Flask đang chạy ở viewport desktop hoặc responsive mobile. Ảnh này chỉ minh họa bước triển khai, không thay thế metric trên test set.

Cách đọc: Đọc từ phần tiêu đề notebook xuống execution count, mã lệnh, output; với biểu đồ đọc tên trục, thang đo, chú giải và xu hướng giữa các epoch hoặc nhóm.

Quan sát: Ảnh cho thấy cell đã được kernel thực thi và output được sinh từ artifact thật của bài toán, không phải ảnh code dựng riêng.

Giải thích chi tiết: Đối chiếu giá trị output với metadata.json và results.json. Nếu train loss giảm nhưng validation loss tăng, mô hình có dấu hiệu overfitting; confusion matrix phải đọc theo từng lớp; hồi quy cần đọc đồng thời MAE, RMSE và R^2.

Giới hạn diễn giải: một lần chia dữ liệu và seed 42 chưa đủ để khẳng định mô hình tổng quát tối ưu. Kết quả trong ảnh chỉ đại diện cho protocol, test set và artifact đã lưu của lần chạy này.